

MikroTik Python Script

Running on Windows via WSL 1

Technical Setup Guide · MikroTik Automation · WSL 1 · VcXsrv

1. Introduction

This document provides a step-by-step guide for running a Linux-based Python script (e.g., using Mactelnet for MikroTik automation) on a Windows machine using Windows Subsystem for Linux (WSL 1). Since certain networking tools only function in a Linux environment, WSL enables seamless execution without requiring a full Linux installation.

2. System Requirements

- › Windows 10 or later
- › Administrator access
- › Internet connection
- › Python 3 installed in WSL
- › Mactelnet (if required by the script)

3. Installing WSL 1

1. Open PowerShell as Administrator.
2. Run the following command to install WSL version 1:

```
wsl --install --version 1
```

i NOTE WSL 1 is required for networking tools like Mactelnet that need direct access to the host network interface. WSL 2 uses a virtualized adapter and may not be compatible.

3. Verify WSL 1 is set as the default version:

```
wsl --set-default-version 1
```

4. Restart your computer if prompted.
5. Open the installed Linux distribution (e.g., Ubuntu).
6. Set up your Linux username and password.

4. Configuring Windows Firewall

Run the following commands in PowerShell as Administrator to allow MikroTik RouterOS management traffic through the Windows Firewall:

7. Allow RouterOS management ports over UDP (MAC-Telnet and Neighbor Discovery):

```
New-NetFirewallRule -DisplayName "RouterOS Management (UDP)" -Direction Inbound -  
Protocol UDP -LocalPort 5678, 20561 -Action Allow
```

8. Allow RouterOS management ports over TCP (SSH and Telnet):

```
New-NetFirewallRule -DisplayName "RouterOS Management (TCP)" -Direction Inbound -
Protocol TCP -LocalPort 22, 23 -Action Allow
```

Port reference:

- > UDP 5678 — MikroTik Neighbor Discovery Protocol (MNDP)
- > UDP 20561 — MAC-Telnet
- > TCP 22 — SSH
- > TCP 23 — Telnet

5. Opening the WSL Terminal

To access the Linux environment, open PowerShell and run:

```
wsl
```

This will launch your default Linux distribution.

6. Configuring Passwordless sudo

Problem

When executing scripts via Windows shortcuts, `sudo` cannot prompt for a password because the process runs in the background.

Solution

Allow your user to execute Python with `sudo` without requiring a password.

Steps

9. Open the WSL terminal.
10. Run:

```
sudo visudo
```

11. Scroll to the bottom of the file and add the following two lines:

```
Defaults!/usr/bin/python3 env_keep += "DISPLAY" charc ALL=(ALL) NOPASSWD:
/usr/bin/python3
```

Line breakdown:

- > `Defaults!/usr/bin/python3 env_keep += "DISPLAY"` — Preserves the `DISPLAY` variable when running Python with `sudo` (required for GUI/Tkinter windows to appear)
- > `charc ALL=(ALL) NOPASSWD: /usr/bin/python3` — Allows the user `charc` to run Python with `sudo` without a password prompt

i NOTE Replace 'charc' with your actual Linux username. This restricts passwordless sudo access to Python only, improving security.

7. Downloading the Project Files

The preflight automation script and all necessary supporting files are packaged in a ZIP file hosted on Google Drive.

Step 1 — Download the ZIP

12. Open a browser and go to the following Google Drive link:

```
https://drive.google.com/file/d/1mfj834rMdZBt5ETzkjitXY1\_wIZlVr5a/view?usp=sharing
```

13. Click the **Download** button (top-right area of the Google Drive page).

14. Save the ZIP file to a location you can easily find, such as your **Downloads** folder or **Desktop**.

Step 2 — Extract the ZIP

15. Navigate to where you saved the ZIP file.

16. Right-click the ZIP file → select **Extract All...** (or use a tool like 7-Zip or WinRAR).

17. Choose a destination folder. A good location is inside your **Documents** folder, for example:

```
C:\Users\charc\Documents\Auto_Preflight_Python\
```

18. Click **Extract**.

Step 3 — Locate the Script and Copy the Path

You need the exact folder path where `preflight_integrated2.py` is located — this will be used in the Windows shortcut command.

19. Open the extracted folder in File Explorer.

20. Navigate inside until you can see `preflight_integrated2.py`.

21. Click the address bar at the top of File Explorer to reveal the full path, then copy it.

i NOTE Example path: `C:\Users\charc\Documents\Auto_Preflight_Python\` — You will need to convert this to WSL format in the shortcut: `/mnt/c/Users/charc/Documents/Auto_Preflight_Python/`

Windows path → WSL path conversion:

- › Replace `C:\` with `/mnt/c/`
- › Replace all backslashes `\` with forward slashes `/`

8. Setting a Static IP on Windows Ethernet

For reliable communication with the MikroTik router, the Windows Ethernet adapter must be set to a static IP address. MikroTik devices typically use the `192.168.88.0/24` subnet by default.

i NOTE This sets your PC's Ethernet adapter to a fixed IP of `192.168.88.100`, which places it on the same subnet as the MikroTik router's default address (`192.168.88.1`).

Method 1 — Using Windows Settings (GUI)

22. Press **Win + R**, type `ncpa.cpl`, and press **Enter** to open Network Connections.

23. Right-click your **Ethernet adapter** → select **Properties**.

24. Double-click **Internet Protocol Version 4 (TCP/IPv4)**.

25. Select **Use the following IP address** and enter the values below:

```
IP address:      192.168.88.100 Subnet mask:      255.255.255.0 Default gateway:
192.168.88.1
```

26. Under **Use the following DNS server addresses**, enter:

```
Preferred DNS: 192.168.88.1
```

27. Click **OK** → **OK** to save and close.

Method 2 — Using PowerShell (Command Line)

Run the following commands in PowerShell as Administrator:

28. Identify your Ethernet adapter name:

```
Get-NetAdapter
```

Look for the **Name** of your Ethernet interface (e.g., **Ethernet** or **Ethernet 2**).

29. Remove any existing IP configuration:

```
Remove-NetIPAddress -InterfaceAlias "Ethernet" -Confirm:$false
```

30. Set the static IP address:

```
New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress 192.168.88.100 -PrefixLength
24 -DefaultGateway 192.168.88.1
```

31. Set the DNS server:

```
Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses 192.168.88.1
```

Replace **Ethernet** with the actual name of your adapter as shown by **Get-NetAdapter**.

i NOTE After setting the static IP, verify connectivity to the MikroTik router by running: ping 192.168.88.1 in PowerShell or CMD.

9. Creating a Windows Shortcut

This allows you to launch the script with a double-click from your Desktop.

32. Right-click on Desktop → New → Shortcut

33. Enter the following command:

```
wsl.exe -d Ubuntu -u charc bash -c "export DISPLAY=:0; cd
/mnt/c/Users/charc/Documents/Auto_Preflight_Python/ && sudo python3
preflight_integrated2.py; exec bash"
```

Command breakdown:

- > **-d Ubuntu** — targets the Ubuntu WSL distribution
- > **-u charc** — runs as the Linux user **charc**
- > **export DISPLAY=:0** — sets the display for GUI output via XLaunch
- > **/mnt/c/Users/charc/Documents/Auto_Preflight_Python/** — navigates to the script directory (copied from Section 7)
- > **sudo python3 preflight_integrated2.py** — runs the script with elevated privileges
- > **exec bash** — keeps the terminal window open after the script finishes

⚙ **What to change for your setup:**

- › **charc** (in `-u charc`) — Replace with your actual WSL Linux username (created during Ubuntu setup)
- › **charc** (in the path) — Replace with your Windows username. Windows paths map to WSL as: `C:\Users\YourName\Folder` → `/mnt/c/Users/YourName/Folder`
- › **preflight_integrated2.py** — Replace with your actual Python script filename

34. Click **Next**, provide a shortcut name, then click **Finish**.

10. Setting Up XLaunch (VcXsrv) with WSL 1

To run graphical Linux applications (such as Tkinter-based Python scripts) on Windows, you need an X Server. Since WSL 1 shares the same network stack as Windows, this is simpler than WSL 2 — no complex IP routing required.

Step 1 — Install VcXsrv

35. Download **VcXsrv Windows X Server** from SourceForge.

36. Install it with the default settings.

Step 2 — Configure XLaunch (Windows Side)

Run XLaunch from your Start menu and follow these steps:

37. **Select display settings:** Choose *Multiple windows* and leave the Display number at `0`. Click **Next**.

38. **Select how to start clients:** Choose *Start no client*. Click **Next**.

39. **Extra settings** (most important):

- › Check *Clipboard and Primary Selection*.

⚠ CRITICAL Check *Disable access control*. If skipped, Ubuntu will be denied permission to display windows on your desktop.

40. **Finish configuration:** Click *Save configuration* and save the file as `config.xlaunch` somewhere easy to find (e.g., your Desktop). Click **Finish**.

Step 2b — Add XLaunch to Windows Startup

To make XLaunch start automatically every time Windows boots, place the saved `.xlaunch` file in the Windows Startup folder.

41. **Open the Startup folder:** Press **Win + R**, type the following and press **Enter**:

```
shell:startup
```

This opens the current user's Startup folder in File Explorer.

42. Copy `config.xlaunch` into this Startup folder.

43. Close the folder. **XLaunch will now start automatically in the background every time you log into Windows.**

i NOTE To verify, restart your PC and check the system tray (bottom-right corner) for the VcXsrv icon. If it appears, XLaunch is running and ready.

Step 3 — Configure Ubuntu (WSL 1 Side)

Tell your Linux terminal to use the X Server you just started.

44. Install a test app and Tkinter:

```
sudo apt update sudo apt install x11-apps python3-tk -y
```

45. **Set the DISPLAY variable:** Since WSL 1 runs on the same machine as Windows, use the local display address:

```
export DISPLAY=:0
```

46. **Make it permanent:** Open your bash profile with nano to avoid re-typing this every session:

```
nano ~/.bashrc
```

Scroll to the bottom of the file and add the following line:

```
export DISPLAY=:0
```

Save and exit: Press **Ctrl + O** to save → **Enter** to confirm → **Ctrl + X** to exit nano.

Then reload the profile to apply changes immediately:

```
source ~/.bashrc
```

Step 4 — Verify the Setup

Run the classic X11 test application to confirm everything is working:

```
xeyes
```

A small pair of eyes should appear on your Windows desktop and follow your cursor. If they appear, your graphical environment is correctly configured and Tkinter scripts will work.

11. Before Running the Script — Pre-Flight Checklist

Before double-clicking the shortcut, make sure everything below is in order. The script will fail silently if any of these are not set up correctly.

☑ Checklist

- › **WSL 1 is installed and Ubuntu is set up** (verify by opening PowerShell and running `ws1 --list --verbose`, confirm VERSION is 1)
- › **Passwordless sudo is configured** (verify by opening WSL and running `sudo python3 --version` — it should not ask for a password)
- › **Project files are extracted** (confirm `preflight_integrated2.py` exists in your specified folder)
- › **Ethernet is set to static IP** (verify by running `ping 192.168.88.1` in PowerShell — you should get a reply from the MikroTik router)
- › **Windows Firewall rules are added** (UDP 5678, 20561 and TCP 22, 23 are allowed inbound)
- › **XLaunch (VcXsrv) is running** (check the system tray at the bottom-right — the VcXsrv icon should be visible)
- › **DISPLAY=:0 is set in ~/.bashrc** (verify by opening WSL and running `echo $DISPLAY` — it should output `:0`)

- › **MikroTik router is connected via Ethernet** (the Ethernet cable should be plugged in between your PC and the MikroTik device)

⚠ CRITICAL Do NOT run the shortcut if XLaunch is not running or the Ethernet static IP is not set — the script window will open but the GUI will fail to display and router communication will not work.

12. Running the Script

Once all items in the pre-flight checklist are confirmed, you are ready to launch.

Simply double-click the shortcut you created on your Desktop. This will automatically:

- › Launch the Ubuntu WSL environment
- › Set the DISPLAY variable for GUI output via XLaunch
- › Navigate to the script directory
- › Run `preflight_integrated2.py` with elevated privileges (no password prompt)
- › Keep the terminal window open after the script finishes

i NOTE If the GUI window does not appear, make sure XLaunch is running (check the system tray) and that `DISPLAY=:0` is set in your `~/.bashrc`.